

BÀI THỰC HÀNH: CẤU TRÚC DỮ LIỆU CÂY (TREE)

1. MỤC TIÊU BÀI THỰC HÀNH

- Hiểu được cấu trúc dữ liệu Cây và đặc biệt là Cây nhị phân tìm kiếm (BST).
- Cài đặt cây nhị phân tìm kiếm bằng C++.
- Thực hiện các thao tác cơ bản: thêm, duyệt, tìm kiếm, xóa node.
- Ứng dụng cây để giải bài toán thực tế như danh bạ, biểu thức, sắp xếp.

2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

Ví dụ 1: Khai báo Node và thêm phần tử vào cây

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = right = nullptr;
    }
};

Node* insert(Node* root, int val) {
    if (root == nullptr) return new Node(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

void inorder(Node* root) {
    if (root != nullptr) {
        inorder(root->left);
        cout << root->data << " ";
    }
}
```

```

        inorder(root->right);
    }
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);
    cout << "Duyệt cây theo LNR (inorder): ";
    inorder(root);
    return 0;
}

```

Ví dụ 2: Tìm kiếm một phần tử trong cây

```

bool search(Node* root, int key) {
    if (root == nullptr) return false;
    if (root->data == key) return true;
    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main() {
    Node* root = nullptr;
    root = insert(root, 10);
    insert(root, 5);
    insert(root, 15);
    int x = 15;
    if (search(root, x))
        cout << x << " được tìm thấy trong cây." << endl;
    else
        cout << x << " không có trong cây." << endl;
    return 0;
}

```

Ví dụ 3: Xóa một phần tử trong cây BST

```
Node* findMin(Node* root) {
    while (root->left != nullptr)
        root = root->left;
    return root;
}

Node* deleteNode(Node* root, int key) {
    if (root == nullptr) return root;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Gọi trong main():
root = deleteNode(root, 30);
cout << "Sau khi xóa 30, duyệt cây: ";
inorder(root);
```

3. NỘI DUNG BÀI THỰC HÀNH

Bài 1: Tạo cây nhị phân tìm kiếm và duyệt cây

- 1. Thêm phần tử vào cây

- 2. Duyệt cây theo thứ tự LNR (Inorder)
- 3. Duyệt cây theo NLR (Preorder)
- 4. Duyệt cây theo LRN (Postorder)
- 5. Thoát

Bài 2: Tìm kiếm trong cây

- Nhập giá trị từ bàn phím và cho biết phần tử có trong cây hay không.

Bài 3: Xóa một phần tử trong cây

- Cho phép người dùng nhập vào phần tử cần xóa và in lại cây theo thứ tự LNR.

Bài 4 (nâng cao): Đếm số node, chiều cao cây, số lá

- Viết các hàm: countNodes, countLeafNodes, height để tính toán các thông số này.

Bài 5 (ứng dụng): Quản lý danh bạ theo tên

- Mỗi node là một người với Họ tên và SĐT.
- BST sắp xếp theo tên.
- Thêm, tìm kiếm, xóa người theo tên.